

Text Large Model + Robotic Arm Control

Function Description

After the program runs, input a sequence of action commands through the terminal. The large model will plan the action functions corresponding to the action commands and execute all commands in sequence.

Startup

Open the terminal and enter the following command:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Then input the commands to control the robotic arm in the text_chat terminal. Currently, the following types of robotic arm control commands are available:

- Individual servo control: Set servo x to y degrees, where x ranges from 1-6 and y ranges from 0-180, for example, set servo 1 to 120 degrees.
- Individual robotic arm action control: Robotic arm up/down/left/right, gripping posture, tracking posture, for example, set the robotic arm to gripping posture, robotic arm extend upward;
- Control robotic arm motion action groups: Robotic arm dance/clap/nod/shake head, for example, please nod your head;
- Change robotic arm end position: Robotic arm adjust up/down/left/right/forward/backward by x centimeters, for example, adjust the robotic arm upward by 3 centimeters.

Freely combine robotic arm control commands. The large model will plan the corresponding action groups for each command. You can refer to the following content and enter it in the text_chat terminal:

First, adjust the robotic arm upward by 3 centimeters, wait 3 seconds, then extend the robotic arm downward, stay for 3 seconds, then have the robotic arm shake its head, pause for 3 seconds, and finally set servo 1 to 150 degrees.

```
root@yahboom:~/LargeModel_ws# ros2 run text_chat text_chat
user input: First, adjust the robotic arm upward by 3 centimeters, wait 3 seconds, then extend the robotic arm downward, stay for 3 seconds, then have the robotic arm shake its head, pause f
or 3 seconds, and finally set servo 1 to 150 degrees.
okay@, let me think for a moment... /[INFO] [1775114395.239738774] [text_chat_node]: "action": ["arm_move('up', 3)"], "response": Alright, I'm going to adjust my arm and do a little dance
for you!
user input: [INFO] [1775114405.567502033] [text_chat_node]: "action": ['wait(3)'], "response": I've moved my arm up by 3 centimeters. Now, I'll wait for 3 seconds before moving down.
[INFO] [1775114413.89478744] [text_chat_node]: "action": ["arm_move('down', 3)"], "response": Time's up! Now I'm moving my arm downward.
[INFO] [1775114424.344947426] [text_chat_node]: "action": ['wait(3)'], "response": I've moved my arm down and will stay here for 3 seconds.
[INFO] [1775114432.313840053] [text_chat_node]: "action": ['arm_shake()'], "response": Alright, time to shake my head a bit!
[INFO] [1775114444.464986462] [text_chat_node]: "action": ['adjust_joint(1, 150)'], "response": I just shook my head like a little robot dancer! Now, I'll adjust servo 1 to 150 degrees.
[INFO] [1775114450.946995997] [text_chat_node]: "action": ['finishtask()'], "response": All done! I've adjusted servo 1 to 150 degrees. That was a fun little sequence!
```

Core Code Analysis

Let's mainly look at the adjust_joint function for setting servo angles. Other functions have been analyzed in the source code analysis of the first lesson [AI Model - Text Version\1. Semantic Understand Instruction Following]. They will not be repeated here.

The adjust_joint function source code path is:

LargeModel_ws/src/largemodel/largemodel/action_service.py

```
#Two parameters are passed in: joint_id is the servo number, with values ranging from 1-6,
and angle is the angle value, with values ranging from 0-180
def adjust_joint(self,joint_id,angle):
    #Directly communicate with the top-level driver board, sending the servo value and angle
    value to be set
    Arm.Arm_serial_servo_write(int(joint_id), int(angle), 1000)
    time.sleep(1.0)
    if not self.combination_mode and not self.interrupt_flag:
        self.action_status_pub("adjust_joint_done")
```